Deleting through a base pointer is undefined behaviour.

Unless the base destructor is virtual.

```
Base* p = new Derived;
delete p;
```

It compiles without a warning. It usually runs.

swipe →

What actually happens

```
struct Base {
    ~Base() { puts("~Base"); }
};

struct Derived : Base {
    int* data = new int[100];
    ~Derived() { delete[] data; puts("~Derived"); }
};
```

STEP	OUTPUT
1	`~Base`
2	**(nothing)**

I ran it: `~Derived` never runs, so `data` leaks. The standard calls it undefined; this is what typical compilers do.

The fix: one keyword

```
struct Base {
    virtual ~Base() { puts("~Base"); }
};
```

STEP	OUTPUT
1	`~Derived`
2	`~Base`

The destructor is picked by the **real type**, like any virtual function.

Derived first, then Base. Nothing else changes in `Derived`.

When do you need it?

YOUR CLASS	DESTRUCTOR
meant as a base, deleted through a `Base*`	**public virtual**
a base that is never deleted through a base pointer	`protected`, not virtual
not meant as a base	nothing special

A simple rule: **if a class has any `virtual` function, give it a virtual destructor.**

Turn on `-Wnon-virtual-dtor` and the compiler will remind you.

What it costs

```
struct Plain {
    int x;
};
struct V {
    virtual ~V() = default;
    int x;
};

sizeof(Plain)   // 4
sizeof(V)       // 16
```

A pointer to the vtable inside **every object**.

Measured on a 64-bit build.

So do not inherit from `std::string` or `std::vector`.

They have no virtual destructor. Use composition instead.

The one-line habit

```
class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};
```

Polymorphic base class? Add `virtual ~Base() = default;`.

Is `-Wnon-virtual-dtor` on in your build?